

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

Дисциплина:
«Информационная безопасность баз данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
Манипулирование данными в БД на языке SQL

Выполнил:
Студент гр. N3248
Жук Анастасия Валерьевна



Проверил:
Салихов Максим Русланович

Санкт-Петербург
2025 г.

Цель работы: получение навыков манипулирования данными в БД при помощи операторов SQL.

Задачи:

1. Изучить операторы языка SQL, позволяющие манипулировать данными в БД.
2. Выполнить задания, описанные в методическом пособии к лабораторной работе.

Ход работы

1. Создать по крайней мере 3 связанные таблицы. Должны быть определены первичные и внешние ключи для таблиц, т.е. по крайней мере для одной пары таблиц должна быть определена связь 1:M.

```
CREATE TABLE Countries (  
    country_id INTEGER PRIMARY KEY,  
    country_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Groups (  
    group_id INTEGER PRIMARY KEY,  
    group_name VARCHAR(10) NOT NULL UNIQUE,  
    group_size INTEGER NOT NULL  
);
```

```
CREATE TABLE Students (  
    student_id INTEGER PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    student_surname VARCHAR(50) NOT NULL,  
    group_id INTEGER NOT NULL,  
    student_country INTEGER NOT NULL,  
    FOREIGN KEY (group_id) REFERENCES Groups(group_id),  
    FOREIGN KEY (student_country) REFERENCES Countries(country_id)  
);
```

```
postgres=# CREATE TABLE Countries (country_id INTEGER PRIMARY KEY, country_name  
VARCHAR(50) NOT NULL);  
CREATE TABLE  
postgres=# CREATE TABLE Groups (group_id INTEGER PRIMARY KEY, group_name VARCHAR  
(10) NOT NULL UNIQUE, group_size INTEGER NOT NULL);  
CREATE TABLE  
postgres=# CREATE TABLE Students (student_id INTEGER PRIMARY KEY, student_name V  
ARCHAR(50) NOT NULL, student_surname VARCHAR(50) NOT NULL, group_id INTEGER NOT  
NULL, student_country INTEGER NOT NULL, FOREIGN KEY (group_id) REFERENCES Groups  
(group_id), FOREIGN KEY (student_country) REFERENCES Countries(country_id));  
CREATE TABLE
```

Проверка связей таблиц: \d название_таблицы – выводит всю информацию о таблице.

2. Наполнить таблицы базы данных при помощи операторов INSERT. Каждая таблица должна иметь не менее 5 разных записей.

```
INSERT INTO Countries (country_id, country_name) VALUES  
(1, 'Россия'),  
(2, 'Казахстан'),  
(3, 'Китай'),  
(4, 'Вьетнам'),
```

(5, 'Беларусь');

```
INSERT INTO Groups (group_id, group_name, group_size) VALUES  
(1, 'N201', 30),  
(2, 'N202', 28),  
(3, 'N203', 33),  
(4, 'N204', 35),  
(5, 'N205', 29);
```

```
INSERT INTO Students (student_id, student_name, student_surname, group_id,  
student_country) VALUES  
(1, 'Михаил', 'Иванов', 3, 1),  
(2, 'Анастасия', 'Омарова', 2, 2),  
(3, 'Джу', 'Ян', 1, 3),  
(4, 'Александр', 'Смирнов', 3, 1),  
(5, 'Валерия', 'Яковлева', 5, 1),  
(6, 'Ксения', 'Алексеева', 4, 1),  
(7, 'Алексей', 'Котов', 5, 5),  
(8, 'Ву', 'Хань', 2, 3);
```

```
postgres=# INSERT INTO Countries (country_id, country_name) VALUES (1, 'Россия')  
, (2, 'Казахстан'), (3, 'Китай'), (4, 'Вьетнам'), (5, 'Беларусь');  
INSERT 0 5  
postgres=# INSERT INTO Groups (group_id, group_name, group_size) VALUES (1, 'N20  
1', 30), (2, 'N202', 28), (3, 'N203', 33), (4, 'N204', 35), (5, 'N205', 29);  
INSERT 0 5  
postgres=# INSERT INTO Students (student_id, student_name, student_surname, grou  
p_id, student_country) VALUES (1, 'Михаил', 'Иванов', 3, 1), (2, 'Анастасия', 'О  
марова', 2, 2), (3, 'Джу', 'Ян', 1, 3), (4, 'Александр', 'Смирнов', 3, 1), (5, '  
Валерия', 'Яковлева', 5, 1), (6, 'Ксения', 'Алексеева', 4, 1), (7, 'Алексей', 'К  
отов', 5, 5), (8, 'Ву', 'Хань', 2, 3);  
INSERT 0 8
```

Результат:

```
postgres=# SELECT * FROM Countries;
```

Текущая кодовая страница: 1251

country_id	country_name
------------	--------------

1	Россия
2	Казахстан
3	Китай
4	Вьетнам
5	Беларусь

(5 строк)

```
postgres=# SELECT * FROM Groups;
```

Текущая кодовая страница: 1251

group_id	group_name	group_size
----------	------------	------------

1	N201	30
2	N202	28
3	N203	33
4	N204	35
5	N205	29

(5 строк)

```
postgres=# SELECT * FROM Students;
```

Текущая кодовая страница: 1251

student_id	student_name	student_surname	group_id	student_country
------------	--------------	-----------------	----------	-----------------

1	Михаил	Иванов	3	1
2	Анастасия	Омарова	2	2
3	Джу	Ян	1	3
4	Александр	Смирнов	3	1
5	Валерия	Яковлева	5	1
6	Ксения	Алексеева	4	1
7	Алексей	Котов	5	5
8	Ву	Хань	2	3

(8 строк)

3. Обновить записи в одной таблице на основании записи из другой (между таблицами должна быть связь).

Изменим количество человек в группе, в которой учится Ксения Алексеева:

```
UPDATE Groups SET group_size = 36 WHERE group_id = (SELECT group_id FROM Students WHERE student_name = 'Ксения' AND student_surname = 'Алексеева');
```

```
postgres=# UPDATE Groups SET group_size = 36 WHERE group_id = (SELECT group_id FROM Students WHERE student_name = 'Ксения' AND student_surname = 'Алексеева');  
UPDATE 1
```

Результат:

```
postgres=# SELECT * FROM Groups;
Текущая кодовая страница: 1251
 group_id | group_name | group_size
-----+-----+-----
         1 | N201      |         30
         2 | N202      |         28
         3 | N203      |         33
         5 | N205      |         29
         4 | N204      |         36
(5 строк)
```

4. Удалить несколько записей из одной таблицы на основании информации из другой таблицы.

Удалим китайских студентов из таблицы Students (допустим, они перевелись в другой университет):

```
DELETE FROM Students WHERE student_country = (SELECT country_id FROM Countries WHERE country_name = 'Китай');
```

```
postgres=# DELETE FROM Students WHERE student_country = (SELECT country_id FROM Countries WHERE country_name = 'Китай');
DELETE 2
postgres=# SELECT * FROM Students;
Текущая кодовая страница: 1251
 student_id | student_name | student_surname | group_id | student_country
-----+-----+-----+-----+-----
         1 | Михаил      | Иванов          |         3 |             1
         2 | Анастасия   | Омарова         |         2 |             2
         4 | Александр   | Смирнов         |         3 |             1
         5 | Валерия     | Яковлева        |         5 |             1
         6 | Ксения      | Алексеева       |         4 |             1
         7 | Алексей     | Котов           |         5 |             5
(6 строк)
```

5. Вывести часть столбцов из таблицы.

```
SELECT student_name, student_surname FROM Students;
```

```
postgres=# SELECT student_name, student_surname FROM Students;
Текущая кодовая страница: 1251
 student_name | student_surname
-----+-----
 Михаил      | Иванов
 Анастасия   | Омарова
 Александр   | Смирнов
 Валерия     | Яковлева
 Ксения      | Алексеева
 Алексей     | Котов
(6 строк)
```

6. Вывести несколько записей из таблицы, используя условие ограничения.

Выведем информацию о студентах, чье имя начинается на букву «А»:

```
SELECT * FROM Students WHERE student_name LIKE 'A%';
```

```
postgres=# SELECT * FROM Students WHERE student_name LIKE 'A%';
Текущая кодовая страница: 1251
 student_id | student_name | student_surname | group_id | student_country
-----+-----+-----+-----+-----
          2 | Анастасия   | Омарова         |         2 |                2
          4 | Александр   | Смирнов         |         3 |                1
          7 | Алексей     | Котов          |         5 |                5
(3 строки)
```

7. Сделать декартово произведение двух таблиц.

```
SELECT * FROM Students, Countries;
```

```
postgres=# SELECT * FROM Students, Countries;
Текущая кодовая страница: 1251
 student_id | student_name | student_surname | group_id | student_country | country_id | country_name
-----+-----+-----+-----+-----+-----+-----
          1 | Михаил       | Иванов          |         3 |                1 |           1 | Россия
          2 | Анастасия   | Омарова         |         2 |                2 |           1 | Россия
          4 | Александр   | Смирнов         |         3 |                1 |           1 | Россия
          5 | Валерия     | Яковлева        |         5 |                1 |           1 | Россия
          6 | Ксения      | Алексеева       |         4 |                1 |           1 | Россия
          7 | Алексей     | Котов          |         5 |                5 |           1 | Россия
          1 | Михаил       | Иванов          |         3 |                1 |           2 | Казахстан
          2 | Анастасия   | Омарова         |         2 |                2 |           2 | Казахстан
          4 | Александр   | Смирнов         |         3 |                1 |           2 | Казахстан
          5 | Валерия     | Яковлева        |         5 |                1 |           2 | Казахстан
          6 | Ксения      | Алексеева       |         4 |                1 |           2 | Казахстан
          7 | Алексей     | Котов          |         5 |                5 |           2 | Казахстан
          1 | Михаил       | Иванов          |         3 |                1 |           3 | Китай
          2 | Анастасия   | Омарова         |         2 |                2 |           3 | Китай
          4 | Александр   | Смирнов         |         3 |                1 |           3 | Китай
          5 | Валерия     | Яковлева        |         5 |                1 |           3 | Китай
          6 | Ксения      | Алексеева       |         4 |                1 |           3 | Китай
          7 | Алексей     | Котов          |         5 |                5 |           3 | Китай
          1 | Михаил       | Иванов          |         3 |                1 |           4 | Вьетнам
          2 | Анастасия   | Омарова         |         2 |                2 |           4 | Вьетнам
          4 | Александр   | Смирнов         |         3 |                1 |           4 | Вьетнам
          5 | Валерия     | Яковлева        |         5 |                1 |           4 | Вьетнам
          6 | Ксения      | Алексеева       |         4 |                1 |           4 | Вьетнам
          7 | Алексей     | Котов          |         5 |                5 |           4 | Вьетнам
          1 | Михаил       | Иванов          |         3 |                1 |           5 | Беларусь
          2 | Анастасия   | Омарова         |         2 |                2 |           5 | Беларусь
          4 | Александр   | Смирнов         |         3 |                1 |           5 | Беларусь
          5 | Валерия     | Яковлева        |         5 |                1 |           5 | Беларусь
          6 | Ксения      | Алексеева       |         4 |                1 |           5 | Беларусь
          7 | Алексей     | Котов          |         5 |                5 |           5 | Беларусь
(30 строк)
```

SELECT * FROM Students CROSS JOIN Countries;

```
postgres=# SELECT * FROM Students CROSS JOIN Countries;
Текущая кодовая страница: 1251
```

student_id	student_name	student_surname	group_id	student_country	country_id	country_name
1	Михаил	Иванов	3	1	1	Россия
2	Анастасия	Омарова	2	2	1	Россия
4	Александр	Смирнов	3	1	1	Россия
5	Валерия	Яковлева	5	1	1	Россия
6	Ксения	Алексеева	4	1	1	Россия
7	Алексей	Котов	5	5	1	Россия
1	Михаил	Иванов	3	1	2	Казахстан
2	Анастасия	Омарова	2	2	2	Казахстан
4	Александр	Смирнов	3	1	2	Казахстан
5	Валерия	Яковлева	5	1	2	Казахстан
6	Ксения	Алексеева	4	1	2	Казахстан
7	Алексей	Котов	5	5	2	Казахстан
1	Михаил	Иванов	3	1	3	Китай
2	Анастасия	Омарова	2	2	3	Китай
4	Александр	Смирнов	3	1	3	Китай
5	Валерия	Яковлева	5	1	3	Китай
6	Ксения	Алексеева	4	1	3	Китай
7	Алексей	Котов	5	5	3	Китай
1	Михаил	Иванов	3	1	4	Вьетнам
2	Анастасия	Омарова	2	2	4	Вьетнам
4	Александр	Смирнов	3	1	4	Вьетнам
5	Валерия	Яковлева	5	1	4	Вьетнам
6	Ксения	Алексеева	4	1	4	Вьетнам
7	Алексей	Котов	5	5	4	Вьетнам
1	Михаил	Иванов	3	1	5	Беларусь
2	Анастасия	Омарова	2	2	5	Беларусь
4	Александр	Смирнов	3	1	5	Беларусь
5	Валерия	Яковлева	5	1	5	Беларусь
6	Ксения	Алексеева	4	1	5	Беларусь
7	Алексей	Котов	5	5	5	Беларусь

(30 строк)

Существует два запроса для декартова произведения двух таблиц.

- Вывести записи из таблицы на основании условия ограничения, содержащегося в другой таблице.

Выведем информацию только о русских студентах.

SELECT * FROM Students WHERE student_country = (SELECT country_id FROM Countries WHERE country_name = 'Россия');

```
postgres=# SELECT * FROM Students WHERE student_country = (SELECT country_id FROM Countries
WHERE country_name = 'Россия');
Текущая кодовая страница: 1251
```

student_id	student_name	student_surname	group_id	student_country
1	Михаил	Иванов	3	1
4	Александр	Смирнов	3	1
5	Валерия	Яковлева	5	1
6	Ксения	Алексеева	4	1

(4 строки)

- Применить функции агрегирования к выводимым записям (sum, avg, min, max)

Для выполнения этого задания добавим колонку в таблицу Students с возрастом студентов, так как некоторые функции применяются только с числовым данным:

ALTER TABLE Students ADD COLUMN student_age INTEGER;


```
UPDATE Students SET student_age = CASE
    WHEN student_id = 1 THEN 20
    WHEN student_id = 2 THEN 19
    WHEN student_id = 4 THEN 21
    WHEN student_id = 5 THEN 19
    WHEN student_id = 6 THEN 20
    WHEN student_id = 7 THEN 19
END;
```

```
postgres=# UPDATE Students SET student_age = CASE WHEN student_id = 1 THEN 20 WHEN student_id = 2 THEN 19 WHEN student_id = 4 THEN 21 WHEN student_id = 5 THEN 19 WHEN student_id = 6 THEN 20 WHEN student_id = 7 THEN 19 END;
UPDATE 6
postgres=# SELECT * FROM Students;
Текущая кодовая страница: 1251
 student_id | student_name | student_surname | group_id | student_country | student_age
-----+-----+-----+-----+-----+-----
          1 | Михаил      | Иванов          |         3 |                 |          20
          2 | Анастасия   | Омарова         |         2 |                 |          19
          4 | Александр   | Смирнов         |         3 |                 |          21
          5 | Валерия     | Яковлева        |         5 |                 |          19
          6 | Ксения      | Алексеева       |         4 |                 |          20
          7 | Алексей     | Котов           |         5 |                 |          19
(6 строк)
```

Суммарный возраст студентов:

```
SELECT SUM(student_age) AS sum_age FROM Students;
```

```
postgres=# SELECT SUM(student_age) AS sum_age FROM Students;
Текущая кодовая страница: 1251
 sum_age
-----
       118
(1 строка)
```

Максимальный возраст студентов:

```
SELECT MAX(student_age) AS max_age FROM Students;
```

```
postgres=# SELECT MAX(student_age) AS max_age FROM Students;
Текущая кодовая страница: 1251
 max_age
-----
        21
(1 строка)
```

Минимальный возраст студентов:

```
SELECT MIN(student_age) AS min_age FROM Students;
```

```
postgres=# SELECT MIN(student_age) AS min_age FROM Students;
Текущая кодовая страница: 1251
 min_age
-----
      19
(1 строка)
```

Средний возраст студентов:

```
SELECT AVG(student_age) AS avg_age FROM Students;
```

```
postgres=# SELECT AVG(student_age) AS avg_age FROM Students;
Текущая кодовая страница: 1251
   avg_age
-----
19.666666666666667
(1 строка)
```

Для строковых типов данных можно применять MIN и MAX. Они будут возвращать первое и последнее имя по алфавиту соответственно. SUM/AVG не работают со строковыми типами данных, вызовут ошибку.

```
SELECT MIN(student_name) AS first_name_alpha, MAX(student_name) AS
last_name_alpha FROM Students;
```

```
postgres=# SELECT MIN(student_name) AS first_name_alpha, MAX(student_name) AS last_name_alpha FROM Students;
Текущая кодовая страница: 1251
 first_name_alpha | last_name_alpha
-----+-----
Александр        | Михаил
(1 строка)
```

10. Вывести записи из таблицы, используя сортировку от большего к меньшему.

```
SELECT * FROM Students ORDER BY student_name DESC;
```

```
postgres=# SELECT * FROM Students ORDER BY student_name DESC;
Текущая кодовая страница: 1251
 student_id | student_name | student_surname | group_id | student_country | student_age
-----+-----+-----+-----+-----+-----
          1 | Михаил      | Иванов          |         3 |                1 |          20
          6 | Ксения      | Алексеева       |         4 |                1 |          20
          5 | Валерия     | Яковлева        |         5 |                1 |          19
          2 | Анастасия  | Омарова         |         2 |                2 |          19
          7 | Алексей     | Котов           |         5 |                5 |          19
          4 | Александр  | Смирнов         |         3 |                1 |          21
(6 строк)
```

DESC позволяет сортировать от большего к меньшему, так как по умолчанию сортировка производится от меньшего к большему.

11. Вывести записи из таблицы, используя сортировку от меньшего к большему с ограничением количества выводимых строк.

```
SELECT * FROM Students ORDER BY student_surname ASC LIMIT 3;
```

```
postgres=# SELECT * FROM Students ORDER BY student_surname ASC LIMIT 3;
Текущая кодовая страница: 1251
 student_id | student_name | student_surname | group_id | student_country | student_age
-----+-----+-----+-----+-----+-----
          6 | Ксения      | Алексеева      |         4 |                 1 |          20
          1 | Михаил     | Иванов         |         3 |                 1 |          20
          7 | Алексей    | Котов          |         5 |                 5 |          19
(3 строки)
```

12. Произвести агрегирование выводимых записей по одному из полей (group by).

Посчитаем количество студентов в разных группах:

```
SELECT group_id, COUNT(*) AS student_count FROM Students GROUP BY group_id;
```

```
postgres=# SELECT group_id, COUNT(*) AS student_count FROM Students GROUP BY group_id;
Текущая кодовая страница: 1251
 group_id | student_count
-----+-----
          3 |             2
          5 |             2
          4 |             1
          2 |             1
(4 строки)
```

Посчитаем средний возраст по группам:

```
SELECT group_id, AVG(student_age) AS avg_age FROM Students GROUP BY group_id;
```

```
postgres=# SELECT group_id, AVG(student_age) AS avg_age FROM Students GROUP BY group_id;
Текущая кодовая страница: 1251
 group_id | avg_age
-----+-----
          3 | 20.5000000000000000
          5 | 19.0000000000000000
          4 | 20.0000000000000000
          2 | 19.0000000000000000
(4 строки)
```

13. Выполнить запрос, когда табличное выражение представляет собой другой запрос.

```
SELECT MIN(student_age) AS min_age_N203
FROM (
    SELECT student_name, student_surname, student_age
    FROM Students
    WHERE group_id = (SELECT group_id FROM Groups WHERE group_name
= 'N203')
```

) AS group_N203;

```
postgres=# SELECT MIN(student_age) AS min_age_N203 FROM (SELECT student_name, student_surname, student_age
FROM Students WHERE group_id = (SELECT group_id FROM Groups WHERE group_name = 'N203')) AS group_N203;
Текущая кодовая страница: 1251
 min_age_n203
-----
                20
(1 строка)
```

Получили промежуточную таблицу group_N203, а потом в ней нашли минимальный возраст студентов группы N203.

Вывод

Были созданы таблицы в БД и заполнены необходимыми данными. Были проведены манипуляции с ними, описанные в методическом пособии.

Основные заключения по работе:

- Для создания структур используется оператор CREATE;
- Для добавления данных в структуры используется INSERT;
- Для удаления всей таблицы используется команда DROP, а для записей – DELETE;
- Для обновления записи в таблице используется оператор UPDATE;
- Для вывода данных используется SELECT;
- Фильтрация производится через WHERE, сортировка – через ORDER BY, агрегация – через GROUP BY, SUM, AVG, MIN, MAX;
- Работа с подзапросами и декартовым произведением осуществляется через CROSS JOIN;
- Внешние ключи помогают связывать таблицы. Если их не задать, то нет гарантии связи даже в случае, если первичный ключ одной таблицы и столбец другой имеют одинаковое название.